

Fleetbase Orchestrator: Comprehensive Feature & UI/UX Plan

Executive Summary

The Fleetbase Orchestrator is envisioned as an industry-standard, engine-agnostic dispatch and route optimization workbench. Moving beyond simple “allocation,” the Orchestrator will empower dispatchers to manage complex last-mile and haulage operations by intelligently coordinating orders, drivers, vehicles, and routes. This document outlines a comprehensive feature set and UI/UX plan based on research into leading platforms like Spoke Dispatch, OptimoRoute, Bringg, and Locus, as well as the capabilities of open-source optimization engines like VROOM and OR-Tools.

1. Core Capabilities & Engine Integration

To compete with industry leaders, the Orchestrator must support a wide array of Vehicle Routing Problem (VRP) constraints. The underlying architecture already supports pluggable engines (e.g., VROOM), which must be fully leveraged.

Supported VRP Constraints

Based on the capabilities of VROOM and OR-Tools [1] [2], the Orchestrator should expose the following constraints to the user:

- **Capacities:** Multi-dimensional capacity constraints (e.g., weight, volume, pallet count) for both vehicles and orders.
- **Time Windows:** Strict and soft time windows for pickups and deliveries, including multiple time windows per location.
- **Skills & Requirements:** Matching specific order requirements (e.g., refrigeration, hazmat, installation) with vehicle capabilities and driver skills.

- **Multi-Depot:** Support for operations spanning multiple starting and ending locations for vehicles.
- **Pickup and Delivery (PDPTW):** Handling linked tasks where an item must be picked up at location A and delivered to location B within the same route.
- **Driver Working Hours & Breaks:** Enforcing maximum shift durations and mandatory break times.

Engine Agnosticism

The UI must remain engine-agnostic. When a user triggers an optimization run, the Orchestrator translates the UI state (selected orders, available drivers, constraints) into a standardized payload for the `AllocationEngineInterface`, which then delegates to the active engine (e.g., `VroomAllocationEngine`).

2. Order Import & Management

A critical feature of any dispatch platform is the ability to rapidly ingest orders.

Spreadsheet Import (Excel/CSV)

Following patterns from OptimoRoute and modern data ingestion tools [3] [4]:

- **Drag-and-Drop Upload:** A dedicated modal or zone for dropping `.csv` or `.xlsx` files.
- **Intelligent Column Mapping:** The system should attempt to auto-map columns (e.g., “Address”, “Weight”, “Time Window Start”) to Fleetbase order fields, allowing the user to manually correct or confirm the mapping.
- **Geocoding & Validation:** During import, addresses must be geocoded. The UI should present a validation screen highlighting errors (e.g., “Address Not Found”, “Missing Weight”) and allow inline corrections before finalizing the import.
- **Custom Fields:** Support for mapping spreadsheet columns to custom order metadata.

Order Filtering & Grouping

The workbench must allow dispatchers to easily slice and dice the order pool:

- Filter by date range, status (unassigned, assigned, routed), zone, or custom tags.
- Group orders by geographic clusters or required skills.

3. The Orchestrator Workbench UI/UX

The workbench is the heart of the Orchestrator. It requires a dense, information-rich, yet intuitive interface, drawing inspiration from platforms like Bringg and Locus [5] [6].

Layout Structure

The UI should be divided into three primary resizable panels:

1. **Order Pool (Left Panel):** A list of unassigned orders, filterable and sortable. Each order card should display key constraints (time window, weight, skills).
2. **Interactive Map (Center Panel):** A map displaying order locations (color-coded by status or zone), depot locations, and generated routes.
3. **Timeline / Route Planner (Right/Bottom Panel):** A Gantt-chart style timeline showing vehicles/drivers on the Y-axis and time on the X-axis. Generated routes appear as blocks on this timeline.

Interactive Features

- **Drag-and-Drop Routing:** Users must be able to drag an unassigned order from the pool and drop it onto a specific driver's timeline or directly onto a route on the map to manually assign it.
- **Lasso Selection:** On the map, users should be able to draw a polygon to select a group of orders for bulk assignment or optimization.
- **Route Modification:** Dragging stops within a route on the timeline to resequence them, with immediate visual feedback on ETA changes and constraint violations (e.g., turning a time window red if the new sequence makes it late).

The Optimization Flow

1. **Selection:** The dispatcher selects a set of orders and a pool of available drivers/vehicles.

2. **Configuration:** A modal allows tweaking of optimization parameters (e.g., “Minimize Distance” vs. “Balance Workload”, toggle specific constraints).
3. **Execution:** The system runs the optimization engine (e.g., VROOM). A loading state is shown.
4. **Review:** The generated routes are displayed on the map and timeline as “Drafts”. The dispatcher can review KPIs (total distance, estimated cost, utilized capacity).
5. **Commit:** The dispatcher approves the plan, which updates the order statuses and dispatches the routes to the drivers’ mobile apps.

4. Real-Time Monitoring & Analytics

Orchestration doesn’t end when routes are dispatched.

Live Tracking

- The map should display real-time driver locations (breadcrumbs) overlaid on the planned routes.
- Order statuses should update dynamically (e.g., “En Route”, “Completed”, “Failed”).

Exception Management

- Visual alerts for deviations: If a driver is running behind schedule and risks missing a time window, the system should flag the affected orders.
- **Dynamic Re-routing:** The ability to inject a high-priority, last-minute order into an active route, prompting the engine to re-optimize the remaining stops for that driver.

5. Implementation Roadmap for Fleetbase

To bring this vision to life within the Fleetbase ecosystem, the following phased approach is recommended:

Phase 1: Foundation & Import

- Implement the CSV/Excel order import flow with column mapping and geocoding validation.
- Enhance the existing `OrchestratorWorkbench` component to support basic filtering and a split map/list view.

Phase 2: Advanced Constraints & VROOM Integration

- Expand the `AllocationEngineInterface` to support capacities, time windows, and skills.
- Update the UI to allow users to define these constraints on orders and vehicles.
- Ensure the VROOM integration correctly processes these advanced VRP parameters.

Phase 3: Interactive Routing & Timeline

- Implement the Gantt-chart timeline view for drivers and routes.
- Add drag-and-drop support for manual order assignment and route resequencing.

Phase 4: Real-Time & Exceptions

- Integrate live driver tracking into the Orchestrator map.
- Implement visual alerts for ETA deviations and constraint violations.

References

- [1] VROOM Project GitHub Repository. <https://github.com/vroom-project/vroom> [2] Google OR-Tools Vehicle Routing Problem Documentation. <https://developers.google.com/optimization/routing/vrp> [3] OptimoRoute Help Center: Import orders from a spreadsheet. <https://help.optimoroute.com/hc/en-us/articles/27661457359124-Import-orders-from-a-spreadsheet> [4] Bringg Documentation: Plan and Dispatch Routes. <https://help.bringg.com/docs/plan-and-dispatch-routes> [5] Locus Dispatch Planning Software. <https://locus.sh/dispatch-planning-software/> [6] Spoke Dispatch Features. <https://spoke.com/dispatch>